Extended reading — Week 8: A2C, GAE, and TRPO

May 21, 2026

1. Mnih et al. (2016) — Asynchronous Methods for Deep Reinforcement Learning

ICML 2016. arXiv:1602.01783.

Summary. Introduces A3C (Asynchronous Advantage Actor-Critic) and its synchronous twin A2C. The core insight is that multiple actors interacting with the environment in parallel decorrelate the training data, removing the need for replay buffers in on-policy algorithms.

Why it matters for this week. The on-policy actor-critic loop in our mini-project is a direct descendant. "Synchronous A2C" — gather rollouts from N vector envs, compute GAE, do one batch SGD step — is what virtually every modern PPO codebase actually runs.

Annotated key passage.

> *"We use a parallel framework to actively decorrelate the agents' data, in contrast with experience replay."*

This is the conceptual move that made on-policy deep RL practical. With $N = 8$–64 envs, each sampled trajectory step is conditionally independent of the others given the current policy — exactly the iid assumption that vanilla SGD's convergence theory leans on. Replay-buffer methods (DQN, SAC) achieve the same goal via stochasticity over time; A2C does it over instances.

2. Schulman, Moritz, Levine, Jordan, Abbeel (2015) — High-Dimensional Continuous Control Using Generalized Advantage Estimation

ICLR 2016. arXiv:1506.02438.

Summary. The GAE paper. Introduces the λ-mixed advantage estimator as a way to interpolate between Monte-Carlo (low bias, high variance) and one-step TD (high bias, low variance). The empirical sweep on MuJoCo is the source of the $\lambda \in [0.92, 0.99]$ folklore.

Why it matters for this week. Theorem 1 of the paper is Theorem **??** in our notes. The derivation is identical; the only differences are notational (they call δ_t "TD residual" and use a slightly different sign convention on dones).

Annotated key passage.

"$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V$"

Two parameters control two distinct dimensions: γ trades off how far into the future we care, λ trades off bias vs. variance in the advantage estimator. Conflating them is a recurring student error — γ is a property of the problem, λ is a property of the estimator.

3. Kakade (2001) — A Natural Policy Gradient

NeurIPS 2001. (No arXiv ID.)

Summary. Introduces the natural gradient idea into RL. Shows that the steepest-ascent direction in the policy manifold (under the Fisher–Rao metric) is $F^{-1}g$, not g. The paper also derives the compatible-function-approximator condition for A_ϕ that makes the natural gradient identical to the regression coefficient of $\nabla \log \pi$ on $\hat{A}$ — a beautiful result.

Why it matters for this week. This is the theoretical foundation of TRPO. Once you accept that gradient descent must occur in policy space (not parameter space), everything else — KL trust regions, Fisher–vector products, K-FAC, even the choice of β in DPO — falls out.

Annotated key passage.

"$\tilde{\nabla}\eta(\theta) = F(\theta)^{-1}\nabla\eta(\theta).$"

Two characters of notation that took the field a decade to fully appreciate. The Fisher inverse is what makes a parameter-space gradient meaningful in distribution space. The same F^{-1} shows up disguised in K-FAC, in elastic-weight-consolidation, and (with F approximated as a constant identity scaled by $1/(2\beta)$) in DPO.

4. Schulman, Levine, Moritz, Jordan, Abbeel (2015) — Trust Region Policy Optimization

ICML 2015. arXiv:1502.05477.

Summary. TRPO. Combines the natural-gradient idea with a monotonic-improvement guarantee. The paper's Theorem 1 (surrogate improvement bound) is the result that the entire safe-RL and constrained-RL literature builds on.

Why it matters for this week. The TRPO derivation in Chapter 2 of our notes is condensed from this paper. Read sections 3–5 carefully; they contain the only honest reckoning with what the surrogate-improvement bound actually guarantees in finite samples (spoiler: less than you might hope, which is why the line search is essential).

Annotated key passage.

> *"Although the theory of TRPO justifies using a penalty instead of a constraint, the penalty coefficient required to guarantee monotonic improvement was prohibitively small. Hence we use the hard-constraint formulation."*

This is the honest admission that the surrogate bound is loose. The proper choice of C would force step sizes too small to be practical, so we use a hard KL constraint with a tuned $\delta \approx 10^{-2}$. This same compromise is exactly what PPO clip does next week — replace a justified-but-too-conservative bound with a heuristic-but-empirically-better one.

5. Engstrom, Ilyas, Santurkar, Tsipras, Janoos, Rudolph, Madry (2020) — Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO

ICLR 2020. arXiv:2005.12729.

Summary. An empirical autopsy of the PPO-vs-TRPO comparison. The headline result: ∼80% of the apparent advantage of PPO over TRPO comes from code-level details (advantage standardisation, value clipping, orthogonal init, learning-rate annealing) rather than the algorithm itself. With these details added, TRPO matches PPO; without them, neither works well.

Why it matters for this week. This paper is the bridge to next week. PPO is famous as "TRPO that just works", but Engstrom shows that PPO works because of *implementation tricks*, not because of the clip mechanism per se. Next week's lecture leans on this paper for the 37-details checklist.

Annotated key passage.

> *"We find that code-level optimizations have, in many cases, a more significant effect than the choice of objective."*

A sobering finding. The lesson: when you read about a new RL algorithm with a 10% performance bump, ask first whether the comparison was apples-to-apples on *all* the 37 details. Usually not.

Sourced from

`research/deeprl_core_sota.md` §2.2–2.5.